

以太网传输图像实验

1、实验任务：

- 1、学习网口传输原理。
- 2、掌握网口传输显示图像的电路设计。

2、实验概要：

UDP (User Datagram Protocol), 即用户数据报协议, 是一种面向无连接的传输层协议。无连接是指在传输数据时, 数据的发送端和接收端不建立逻辑连接。简单来说, 当一台计算机向另外一台计算机发送数据时, 发送端不会确认接收端是否存在, 就会发出数据, 同样接收端在收到数据时, 也不会向发送端反馈是否收到数据。由于使用 UDP 协议消耗资源小, 通信效率高, 所以通常都会用于音频、视频和普通数据的传输 (如视频会议等) 都会采用 UDP 协议进行传输, 这种情况即使偶尔丢失一两个数据包, 也不会对接收结果产生太大影响。开放系统互联模型 (OSI, Open System Inter Connect), 是国际标准化组织在 1985 年研究的网络互联模型。该体系结构定义了网络互联的七层框架, 物理层-数据链路层-网络层-传输层-会话层-表示层-应用层, 即开放系统互联模型。UDP 和 TCP 是传输层中非常重要的两个协议, 位于 OSI (Open System Interconnection, 开放式系统互联) 参考模型中的第四层 (传输层), 是一种无连接的传输层协议, 提供面向事务的简单不可靠信息传送服务, 位于 IP 协议层 (网络层) 之上。OSI 参考模型如下图 7-1 所示。

OSI七层模型	功能	对应的网络协议
应用层	应用层是网络体系中最高的一层, 也是唯一面向用户的一层, 也可视为为用户提供常用的应用程序, 每个网络应用都对应着不同的协议	HTTP、TFTP, FTP, NFS, WAIS、SMTP
表示层	主要负责数据格式的转换, 确保一个系统的应用层发送的消息可以被另一个系统的应用层读取, 编码转换, 数据解析, 管理数据的解密和加密, 同时也对应用层的协议进行翻译	Telnet, Rlogin, SNMP, Gopher
会话层	负责网络中两节点的建立, 在数据传输中维护计算机网络中两台计算机之间的通信连接, 并决定何时终止通信	SMTP, DNS
传输层	是整个网络关键的部分, 是实现两个用户进程间端到端的可靠通信, 处理数据包的错误等传输问题。是向下通信服务最高层, 向上用户功能最底层。即向网络层提供服务, 向会话层提供独立于网络层的传送服务和可靠的透明数据传输。	TCP, UDP
网络层	进行逻辑地址寻址, 实现不同网络之间的路径选择, IP就在网络层	IP, ICMP, ARP, RARP, AKP, UUCP
数据链路层	物理地址 (MAC地址), 网络设备的唯一身份标识。建立逻辑连接、进行硬件地址寻址, 相邻的两个设备间的互相通信	FDDI, Ethernet, Arpanet, PDN, SLIP, PPP, STP, HDLC, SDLC, 帧中继
物理层	七层模型中的最底层, 主要是物理介质传输媒介 (网线或者是无线), 在不同设备中传输比特, 将0/1信号与电信号或者光信号互相转化	IEEE 802.1A, IEEE 802.2到 IEEE 802

图 7-1

UDP 信息包的标题很短, 只有 8 个字节, 相对于 TCP 的 20 个字节信息包而言 UDP 的额外开销很小。吞吐量不受拥挤控制算法的调节, 只受应用软件生成数据的速率、传输带宽、源端和终端主机性能的限制。UDP 是面向报文的。发送方的 UDP 对应用程序交下来的报文, 在添加首部后就向下交付给 IP 层。既不拆分, 也不合并, 而是保留这些报文的边界, 因此, 应用程序需要选择合适的报文大小。

UDP 通信应用方案功能框图如图 7-2 所示

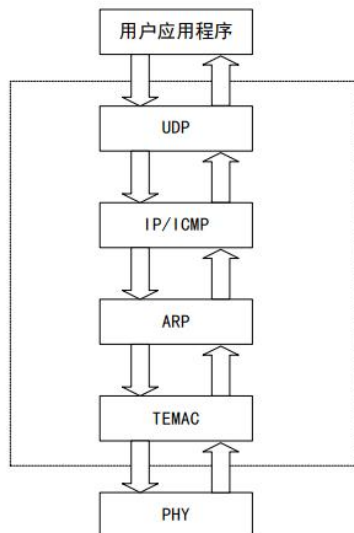


图 7-2

其中用户应用程序为在中间显示传输图像的部分内容，UDP 上层可以与用户应用程序进行数据的接收和发送，下层通过安路的三速以太网 IP 与物理层通信，支持单播、组播和广播的通信方式。用户应用程序和 UDP 通过 FIFO 进行数据的交换。

本方案时钟结构如图所示。从 TEMAC 传来的数据通过异步 fifo 传输给 UDP 模块，两边时钟域如图 7-3 中虚线所示，TEMAC 可以工作在三个速度下，UDP 可以相应调整输入的时钟以适应三种不同的带宽。

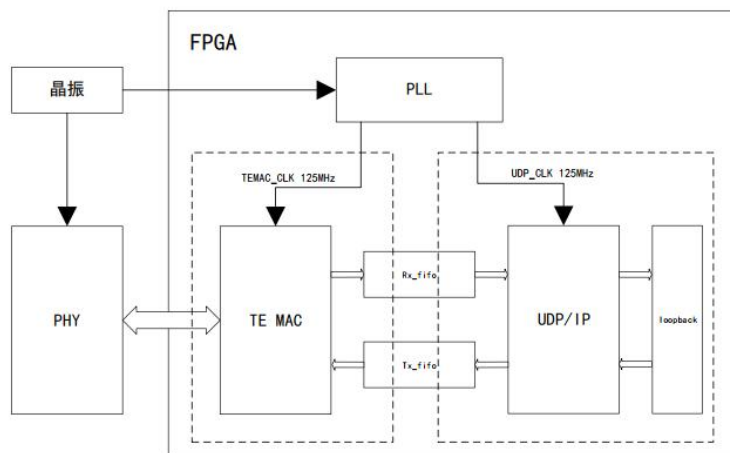


图 7-3

UDP 模块的主要端口如图 7-4 所示

信号名	bits	方向	功能描述
udp_rx_clk	1	input	UDP 接收时钟，千兆模式 125MHz，百兆模式 12.5MHz，十兆模式 1.25MHz。
udp_tx_clk	1	input	UDP 发送时钟，千兆模式 125MHz，百兆模式 12.5MHz，十兆模式 1.25MHz。
reset	1	input	全局复位信号，高电平有效。
udp2app_tx_ready	1	output	UDP 和 APP 之间的握手信号，指示准备接收数据，高电平有效。
udp2app_tx_ack	1	output	UDP 和 APP 之间的握手信号，指示响应接收数据，高电平有效。
app_tx_request	1	input	UDP 和 APP 之间的握手信号，指示请求发送数据，高电平有效。
app_tx_data_valid	1	input	APP 层发送数据有效指示信号，高电平有效。
app_tx_data	8	input	APP 层发送数据。
app_tx_data_length	16	input	APP 层发送数据长度，数据包的字节数。
app_rx_data_valid	1	output	APP 层接收数据有效指示信号，高电平有效。
app_rx_data	8	output	APP 层接收数据。
app_rx_data_length	16	output	APP 层接收数据长度，数据包的字节数。

图 7-4

(1) 其中在 app 模块负责接收 UDP 传输的数据，将数据存储在 6*32768 的伪双口 ram 中（在数据输入前 ram 提前加载 logo.mif 文件），通过 udp 模块的数据以及数据有效位，将 PC 传输的数据写入 ram 中，数据全部写完后将 ram 的写使能拉低，ram 的读使能拉高，通过 app 的子模块 vga_disp 显示接受的数据。

(2) 将 UDP 端口的 app_rx_data, app_rx_data_length, app_rx_data_valid 信号接入到 app 端口当 app_rx_data_vaild 高有效时，ram 数据的地址随时钟加一，同时写入 app_rx_data 的数据。接收数据时端口时序如图 7-5 所示。

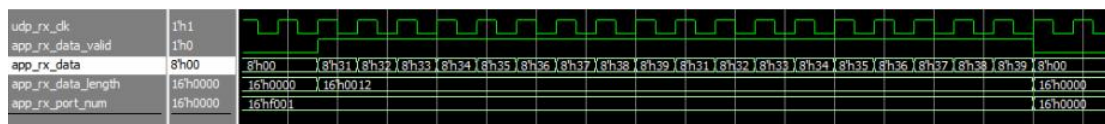


图 7-5

(3) 图像数据的接收采用三段式状态机接收如下图 7-6 所示

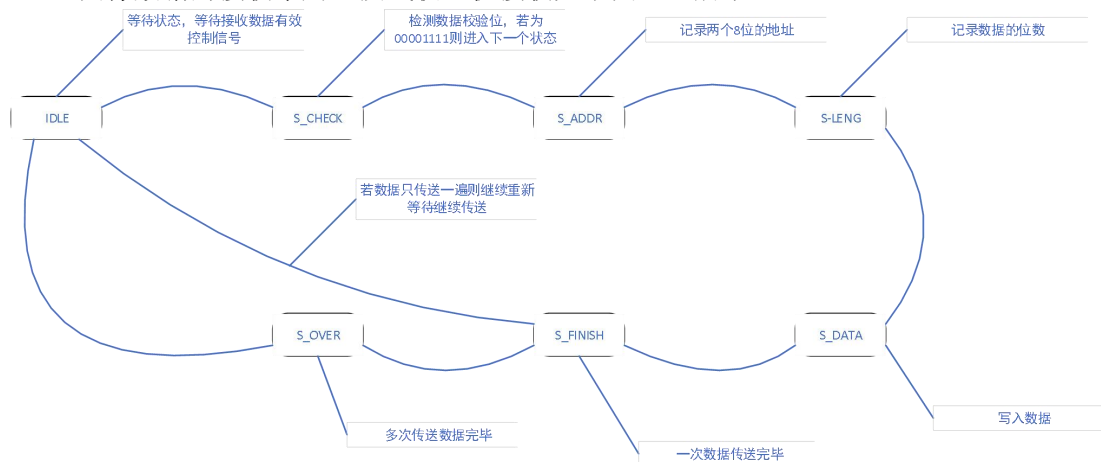


图 7-6

通过该状态机将数据接受并写入 ram 中，当写入完毕后拉高 ram 的读使能，从而进入图像使用模块。

(4) 图像的显示模块可参考 rom 显示图像实验。

(5) PC 传输数据可通过网口工具 NetAssist 进行数据输入。在快捷指令中输入包含的地址校验位以及长度和数据的包并循环发送，在接收区可接收到环回程序送回的接收数据。在 vga 显示器上可观察到想要的颜色图像。

3、引脚约束

```
create_clock -name clk_in -period 40 -waveform {0 20} [get_ports {clk_25}]
create_clock -name phy1_rgmii_rx_clk -period 8 -waveform {0 4} [get_ports {phy1_rgmii_rx_clk}]
create_generated_clock -name {pll_inst_125M_0} -source [get_ports {clk_25}] -master_clock {clk_in}
-multiply_by 5.000 [get_pins {u_clk_gen/u_pll_0/pll_inst.clkc[0]}]
create_generated_clock -name {pll_inst_125M_1} -source [get_ports {clk_25}] -master_clock {clk_in}
-multiply_by 5.000 [get_pins {u_clk_gen/u_pll_0/pll_inst.clkc[1]}]
create_generated_clock -name {pll_inst_12p5M} -source [get_ports {clk_25}] -master_clock {clk_in}
-divide_by 2.000 [get_pins {u_clk_gen/u_pll_0/pll_inst.clkc[2]}]
create_generated_clock -name {pll_inst_25M} -source [get_ports {clk_25}] -master_clock {clk_in}
-divide_by 1.000 [get_pins {u_clk_gen/u_pll_0/pll_inst.clkc[3]}]
```

```
#create_generated_clock -name {clk_1_25_out} -source [get_pins
{u_clk_gen/u_pll_0/pll_inst.clkc[2]]} -master_clock {pll_inst_12p5M} -divide_by 10.000 [get_nets
{u_clk_gen/u_udp_clk_gen_1p25/div_reg3}]
create_generated_clock -name {udp_clk_125m} -add -source [get_pins
{u_clk_gen/u_pll_0/pll_inst.clkc[1]]} -master_clock {pll_inst_125M_1} -divide_by 1.000 [get_nets
{udp_clk}]
create_generated_clock -name {udp_clk_12p5m} -add -source [get_pins
{u_clk_gen/u_pll_0/pll_inst.clkc[2]]} -master_clock {pll_inst_12p5M} -divide_by 1.000 [get_nets
{udp_clk}]
create_generated_clock -name {udp_clk_1p25m} -add -source [get_pins
{u_clk_gen/u_pll_0/pll_inst.clkc[2]]} -master_clock {pll_inst_12p5M} -divide_by 10.000 [get_nets
{udp_clk}]
set_clock_groups -exclusive -group [get_clocks {udp_clk_125m}]
set_clock_groups -exclusive -group [get_clocks {udp_clk_12p5m}]
set_clock_groups -exclusive -group [get_clocks {udp_clk_1p25m}]
set_clock_groups -exclusive -group [get_clocks {phy1_rgmii_rx_clk}]
derive_pll_clocks -instance [get_cells {u_rx_pll/pll_inst}]
```

4、下载验证

首先将电脑 IP 修改为“192.168.240.2”如图 7-7 所示

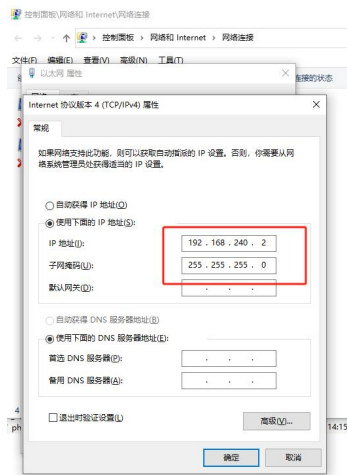


图 7.7

然后连接开发板，网线插在开发板上，网线另一端连接到主机。插好 VGA 线缆，另一端连接显示器，插好下载线打开电源，烧写程序到开发板内，图 7-8

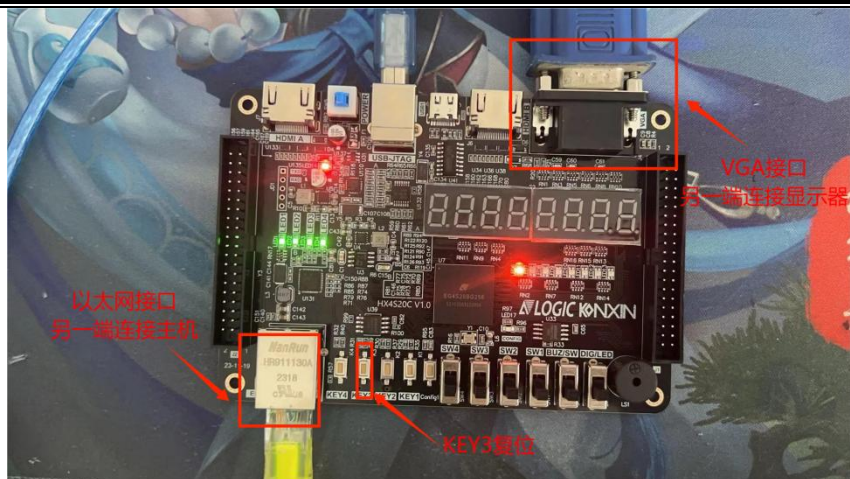


图 7-8

烧写程序到开发板后此时显示屏显示图像如图 7-9 所示

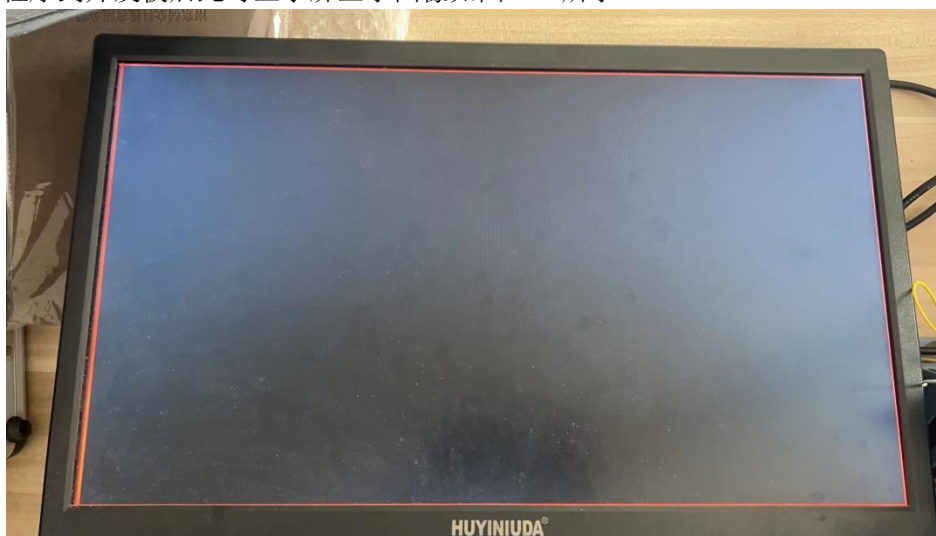


图 7-9

打开网络调试助手，点击批量发送定义，图 7-10。

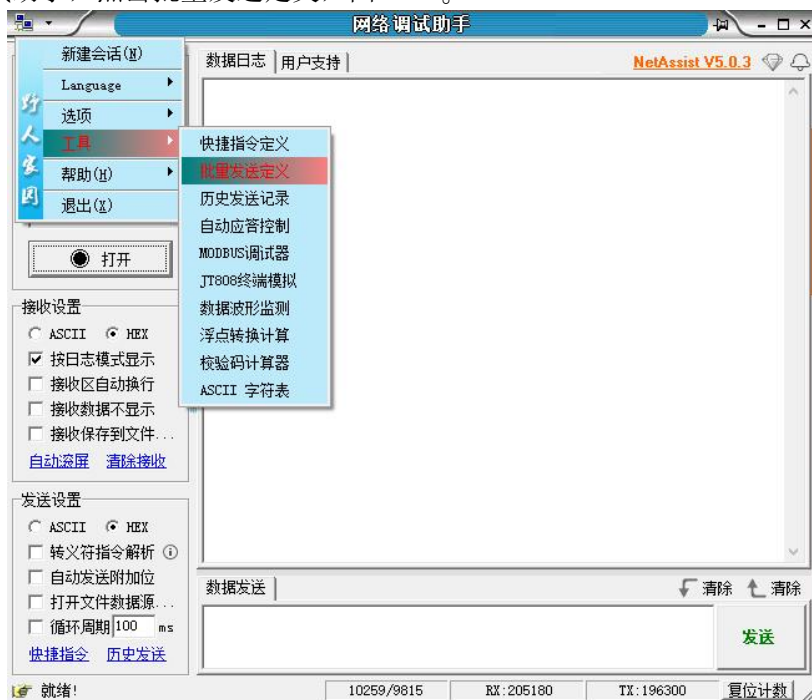


图 7-10

然后导入图像文件，如下图 7-11 所示，图像文件在工程“图像文件”内。

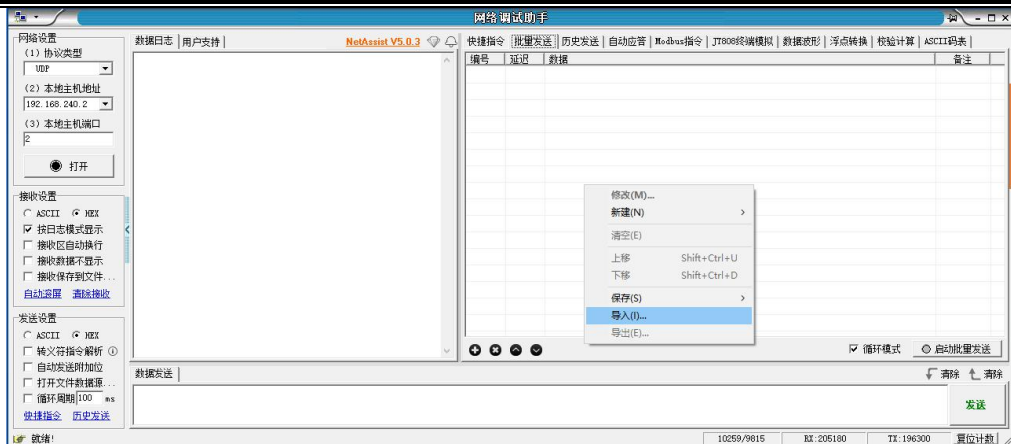


图 7-11

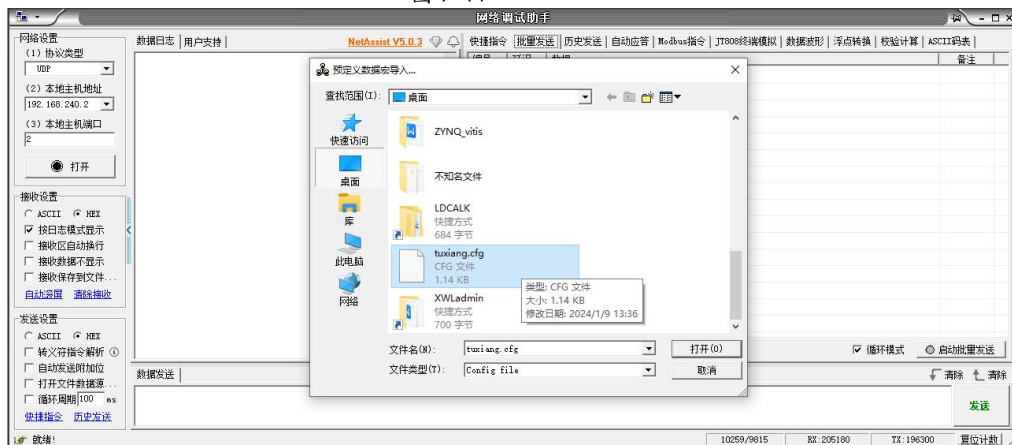


图 7-12

图 7-12，导入 cfg 文件。

图 7-13 成功导入。

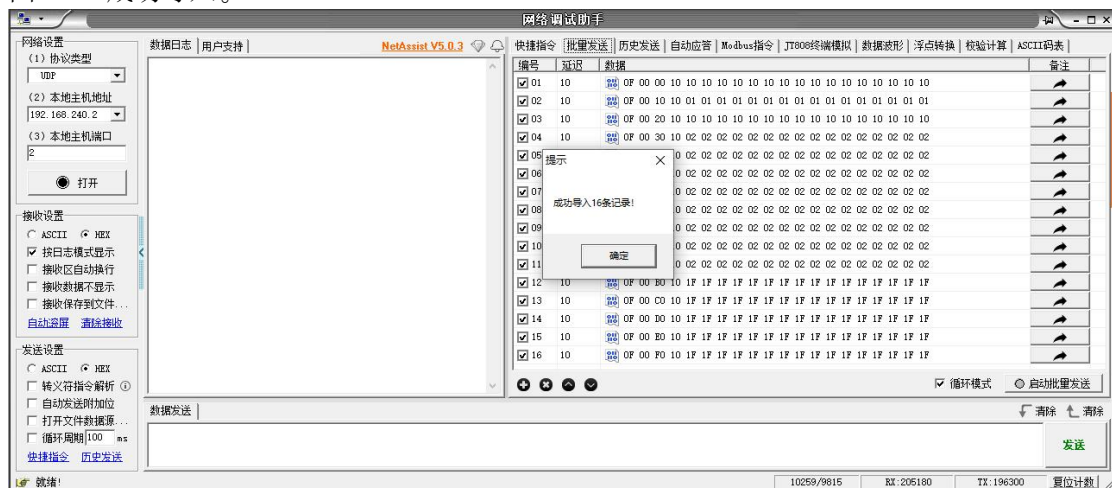


图 7-13

设置远程主机地址后打开端口，如下图 7-14 所示：



图 7-14

开启循环模式，点击批量发送，如下图 7-15 所示

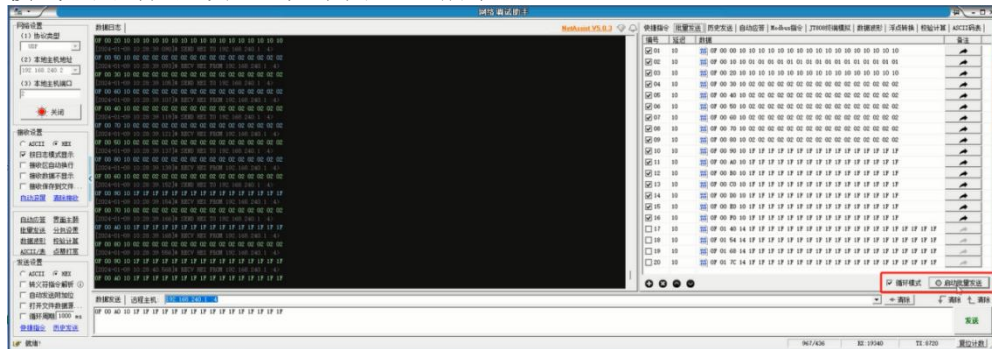


图 7-15

此时可以在显示器上看到，中心出现的蓝色图像，验证成功，其发送的数据可以自行定义。